

Assignment 2

Computer Networks

Name: Kavya Bhand

Class: TY CE-F

Roll No: 45

Line Coding

Line coding is the process of converting binary data into digital signals for transmission over a physical communication medium.

Each bit (0 or 1) is represented by one or more voltage levels according to a predefined encoding rule.

Types of Line Coding

1. Unipolar
 2. Polar
 3. Bipolar
-

Unipolar

In unipolar encoding, only one voltage polarity is used. There are two types in Unipolar Line Coding:

1. Unipolar NRZ (Non Return to Zero)

Bit	Voltage
1	+V
0	0

2. Unipolar RZ (Return to Zero)

Bit	Voltage
1	+V then 0

0	0 then 0
---	----------

Polar

Polar encoding uses both positive and negative voltages. Positive voltage represents one binary value while negative voltage represents the other.

1. Polar NRZ-Level

Bit	Voltage
1	+V
0	-V

2. Polar NRZ-Inversion

Bit	Voltage
1	Change voltage level
0	No change

3. Polar RZ

Bit	Voltage
1	+V then 0
0	-V then 0

Bipolar

Bipolar encoding uses three voltage levels and successive 1s alternate between positive and negative voltages.

1. Bipolar NRZ

Bit	Voltage
1	Alternate between +V and -V
0	0

2. Manchester Encoding

Bit	Voltage
1	+V → -V
0	-V → +V

3. Differential Manchester Encoding

Bit	Voltage
1	No transition at beginning
0	Transition at beginning

Implementation

```
#include <iostream>
using namespace std;

int main()
{
    int choice, subchoice;
    string bits;

    do
    {
        cout << endl;
        cout << "1. Unipolar\n";
        cout << "2. Polar\n";
        cout << "3. Bipolar\n";
        cout << "4. Exit\n";
        cout << "Enter Choice: ";
        cin >> choice;

        if(choice == 4)
            break;

        cout << "Enter Binary Data: ";
        cin >> bits;
```

```

switch(choice)
{
case 1:
    cout << "\n1. NRZ\n2. RZ\n";
    cout << "Enter Choice: ";
    cin >> subchoice;

    if(subchoice == 1)
    {
        cout << "Output: ";
        for(char b : bits)
            cout << (b == '1' ? "+V " : "0 ");
    }
    else if(subchoice == 2)
    {
        cout << "Output: ";
        for(char b : bits)
            cout << (b == '1' ? "[+V 0] " : "[0 0]
");
    }
    break;

case 2:
    cout << "\n1. NRZ-Level\n2. NRZ-Inversion\n3. RZ
\n";

    cout << "Enter Choice: ";
    cin >> subchoice;

    if(subchoice == 1)
    {
        cout << "Output: ";
        for(char b : bits)
            cout << (b == '1' ? "+V " : "-V ");
    }
    else if(subchoice == 2)

```

```

    {
        int level = 1;
        cout << "Output: ";

        for(char b : bits)
        {
            if(b == '1')
                level *= -1;

            cout << (level == 1 ? "+V " : "-V ");
        }
    }
    else if(subchoice == 3)
    {
        cout << "Output: ";
        for(char b : bits)
            cout << (b == '1' ? "[+V 0] " : "[-V 0]");
    }
    break;

case 3:
    cout << "\n1. NRZ (AMI)\n2. Manchester\n3. Differential Manchester\n";
    cout << "Enter Choice: ";
    cin >> subchoice;

    if(subchoice == 1)
    {
        int level = 1;
        cout << "Output: ";

        for(char b : bits)
        {
            if(b == '1')
            {

```

```

        cout << (level == 1 ? "+V " : "-V ");
        level *= -1;
    }
    else
        cout << "0 ";
}
}
else if(subchoice == 2)
{
    cout << "Output: ";

    for(char b : bits)
    {
        if(b == '1')
            cout << "[+V -V] ";
        else
            cout << "[-V +V] ";
    }
}
else if(subchoice == 3)
{
    bool high = true;
    cout << "Output: ";

    for(char b : bits)
    {
        if(b == '0')
            high = !high;

        if(high)
            cout << "[+V -V] ";
        else
            cout << "[-V +V] ";

        high = !high;
    }
}

```

```

    }
    break;

default:
    cout << "Invalid Choice!";
}

cout << "\n";

} while(choice != 4);

return 0;
}

```

```

1. Unipolar
2. Polar
3. Bipolar
4. Exit
Enter Choice: 1
Enter Binary Data: 10110

1. NRZ
2. RZ
Enter Choice: 1
Output: +V 0 +V +V 0

```

```

1. Unipolar
2. Polar
3. Bipolar
4. Exit
Enter Choice: 1
Enter Binary Data: 10110

1. NRZ
2. RZ
Enter Choice: 2
Output: [+V 0] [0 0] [+V 0] [+V 0] [0 0]

```

```

1. Unipolar
2. Polar
3. Bipolar
4. Exit
Enter Choice: 2
Enter Binary Data: 10110

1. NRZ-Level
2. NRZ-Inversion
3. RZ
Enter Choice: 1
Output: +V -V +V +V -V

```

```

1. Unipolar
2. Polar
3. Bipolar
4. Exit
Enter Choice: 2
Enter Binary Data: 10110

1. NRZ-Level
2. NRZ-Inversion
3. RZ
Enter Choice: 2
Output: -V -V +V -V -V

```

```

1. Unipolar
2. Polar
3. Bipolar
4. Exit
Enter Choice: 2
Enter Binary Data: 10110

1. NRZ-Level
2. NRZ-Inversion
3. RZ
Enter Choice: 3
Output: [+V 0] [-V 0] [+V 0] [+V 0] [-V 0]

```

```

1. Unipolar
2. Polar
3. Bipolar
4. Exit
Enter Choice: 3
Enter Binary Data: 10110

1. NRZ (AMI)
2. Manchester
3. Differential Manchester
Enter Choice: 1
Output: +V 0 -V +V 0

```

```

1. Unipolar
2. Polar
3. Bipolar
4. Exit
Enter Choice: 3
Enter Binary Data: 10110

1. NRZ (AMI)
2. Manchester
3. Differential Manchester
Enter Choice: 2
Output: [+V -V] [-V +V] [+V -V] [+V -V] [-V +V]

```

```

1. Unipolar
2. Polar
3. Bipolar
4. Exit
Enter Choice: 3
Enter Binary Data: 10110

1. NRZ (AMI)
2. Manchester
3. Differential Manchester
Enter Choice: 3
Output: [+V -V] [+V -V] [-V +V] [+V -V] [+V -V]

```

